Funciones y Procedimientos

Alumnas:
Viviam Guerein
Leticia Perez
Milagros Fernández

Índice

- 3 Modularidad
- 5 Métodos
- 7 Sintaxis de métodos
- 9 Funciones
- 10 Procedimiento
- 11 Ejemplos (individuales)

¿Que es la Modularidad en programación?

La modularidad consiste en organizar un problema dividiéndola en partes más pequeñas llamadas subprogramas o métodos. Los cuales se van a resolver por separado.

Cuando un programa comienza a ser largo y complejo, no es apropiado tener un único texto con sentencias una tras otra. La razón es que no se comprende bien qué hace el programa debido a que se intenta abarcar toda la solución a la vez.

Programa principal
Módulo A
Módulo B
Módulo C
Módulo D
Módulo E
Módulo F
< d_codinGames />

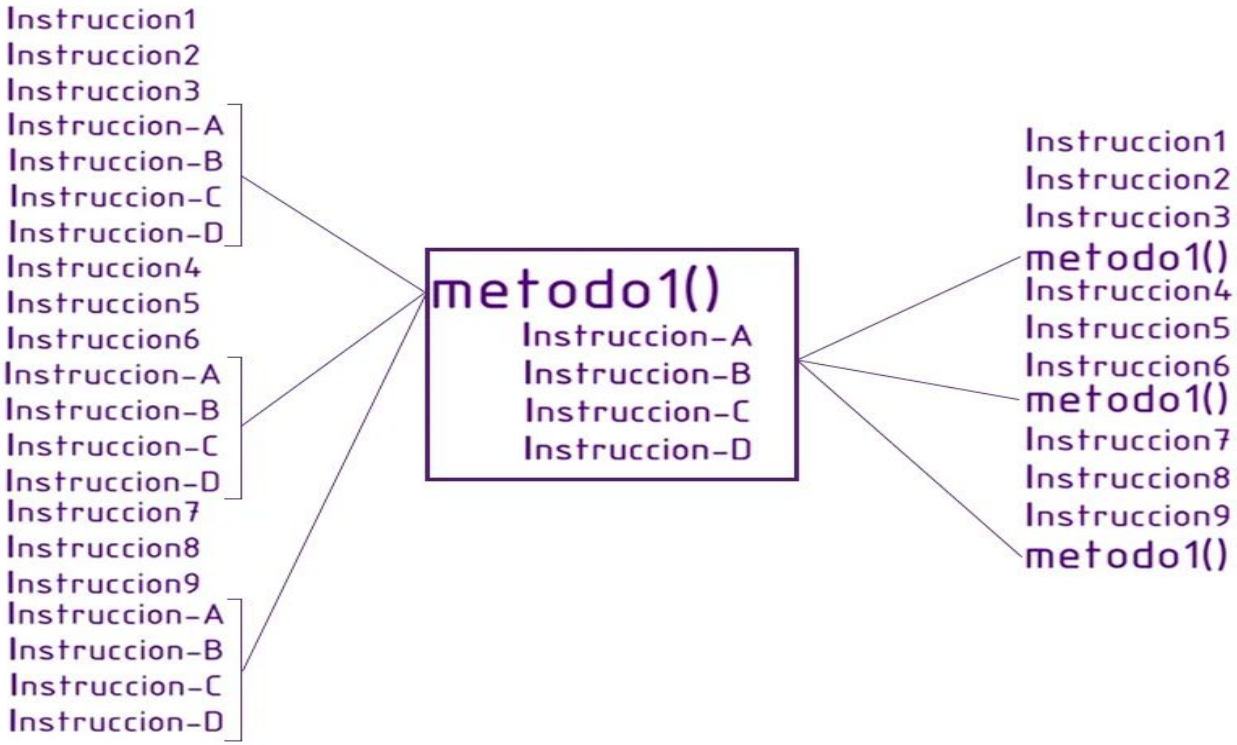
Método

Un **método** es un **bloque de código** que realiza una tarea específica y que puede ser invocado (llamado) desde otras partes del programa.

Un método:

- **Pertenece** a una clase u objeto.
- **Puede recibir** datos de entrada llamados **parámetros**.
- **Puede devolver** un valor (o no devolver nada, usando *void*).
- **Permite** reutilizar código, mejorar la legibilidad y organizar mejor el programa.

Entonces un método es una secuencia ordenada de pasos muy precisos que conducen a la solución de un problema.



Código con redundancias

Código sin redundancias

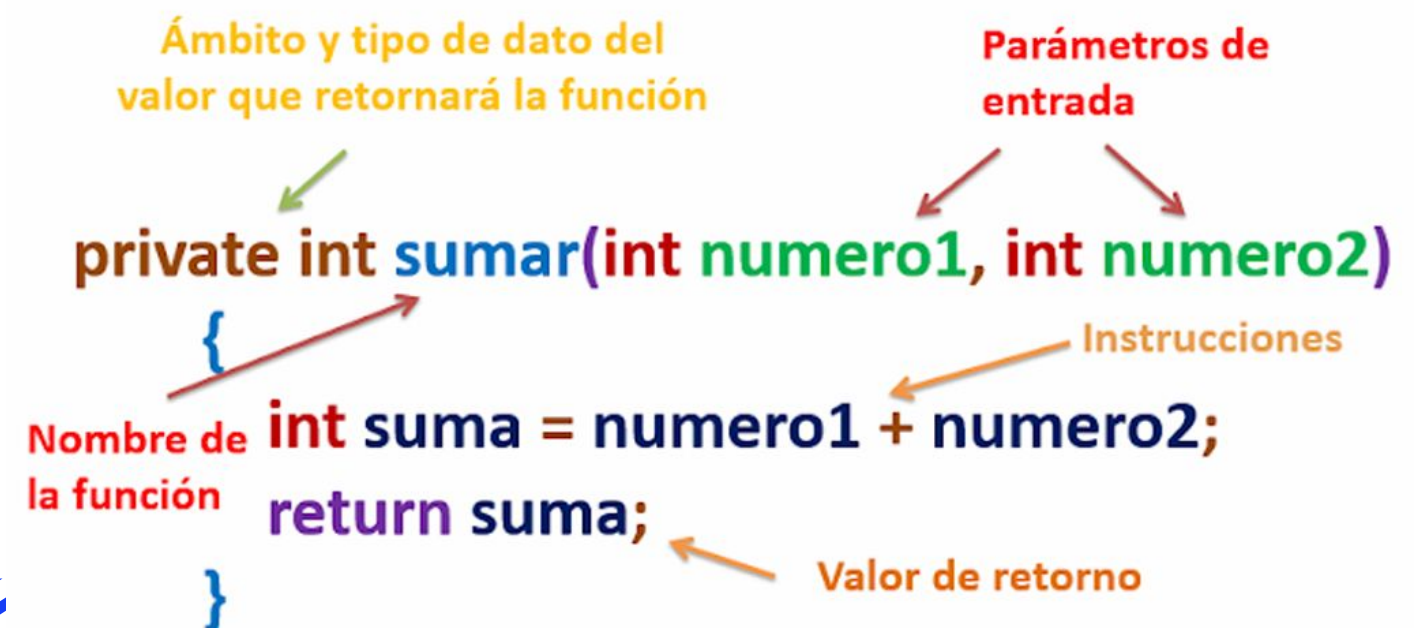
Sintaxis de definición de método

```
[especificadores] tipoRetorno nombre([parámetros]) {
  // instrucciones
  [return valor;]
}
```

- **especificadores**: determinan el tipo de acceso al método.(por ejemplo public - private)
- **tipoRetorno:** tipo de dato que devuelve el método o void si no devuelve ningún valor. (por ejemplo int - double - string - boolean - void)
- **nombre:** identificador del método que se va a utilizar.(por ejemplo identificador del método que se va a utilizar mensaje)
- **parámetros (opcionales):** estos son los datos de entrada que recibe el método para operar con ellos.(por ejemplo números, texto, o vacío. Estos deben ir dentro de paréntesis)
- **return:** se utiliza para retornar un valor. El tipo del valor retornado debe ser igual al tipoRetorno.
- **Bloque {}:** un bloque tiene un único punto de inicio, representado por la llave de inicio. La ejecución de un método termina cuando se llega a la llave final.
- **Sentencia ; :**hace referencia a la terminación de una instrucción.
- **Corchete [ ]** : no ejecutan acciones por si solos, indican un arreglo de texto.

Funciones

Una función es un conjunto de instrucciones que realiza una tarea específica y devuelve un resultado.



Procedimiento

Los procedimientos son básicamente un conjunto de instrucciones que se ejecutan sin retornar ningún valor.

Ámbito de la declaración

Nombre del procedimiento

```
private void limpiar ()
    {
    txtNumero1.setText(null);
    }
```

Instrucciones

Ejemplo 1

```java
public class mainEjemplo {

    // Definimos una función que suma dos números
    public static int sumar(int a, int b) {  1 usage
        return a + b; // devuelve la suma
    }

    public static void main(String[] args) {
        // Llamamos a la función y mostramos el resultado
        int resultado = sumar( a: 1,  b: 2);
        System.out.println("La suma es: " + resultado);
    }
}
```

Ejemplo 2

```java
package procedimiento;

public class EjemploFunciones {
    public static double calcularArea(double base, double altura) {  1 usage
        return (base * altura) / 2; // Fórmula: (base × altura) / 2
    }

    public static void main(String[] args) {
        double resultado = calcularArea( base: 8,  altura: 5);
        System.out.println("El área del triángulo es: " + resultado);
    }
}
```

Ejemplo 3

```
package procedimiento;

public class EjercicioProcedimiento {
    public static  void main(String[] args) {
        String mensaje= "Hola como estás?";
                imprime (mensaje);
    }
            private static void imprime (String mensaje){  1 usage
                System.out.println(mensaje);
        }
}
```

Ejemplo 4

```java
package holamundo;

public class ejemplo {
    public static void main(String[] args) {
            String nombre = "Leticia";
            String apellido = "Perez";

            // Concatenar usando variables
            String mensaje = "Hola " + nombre + " " + apellido + ". ¡Bienvenida al mundo de la programación!.";

            System.out.println(mensaje);
    }
}
```

Ejemplo 5

```java
package holamundo;

public class ejemplo1 {
    public static int multiplicar(int a, int b) {  1 usage
        return a * b;
    }
    public static void main(String[] args) {
        int resultado = multiplicar( a: 4,  b: 5);
        System.out.println("El resultado es: " + resultado);
    }
}
```